



Airflow Summit 2026 – Building a low-cost, scalable Airflow Platform for small teams



Aniruddha Sengupta

Contents



- Introduction
- Data Engineering context
- Our constraints
- Composer Environment optimization
- Airflow best practice
- Logs and metadata
- Environment separation
- Our methods
- Takeaways
- Q&A

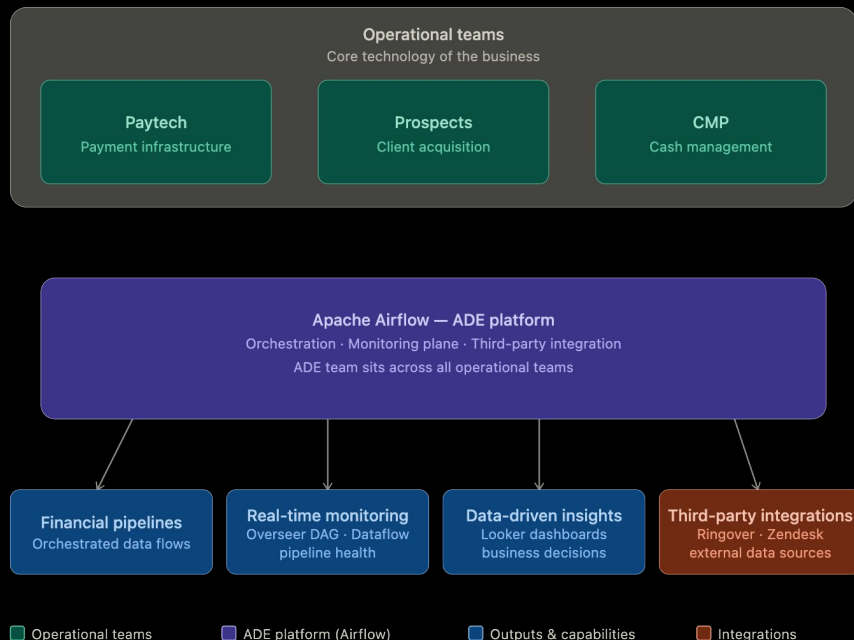
Introduction



- Fintech based in UK (London and Cambridge)
- Manage cash savings across businesses and people across the UK -> make it easier to manage and diversify deposits and find the highest returns.
- Build a digital platform where client cash is managed into savings accounts with the highest interest rates.

INSIGNIS 

Data Engineering Context



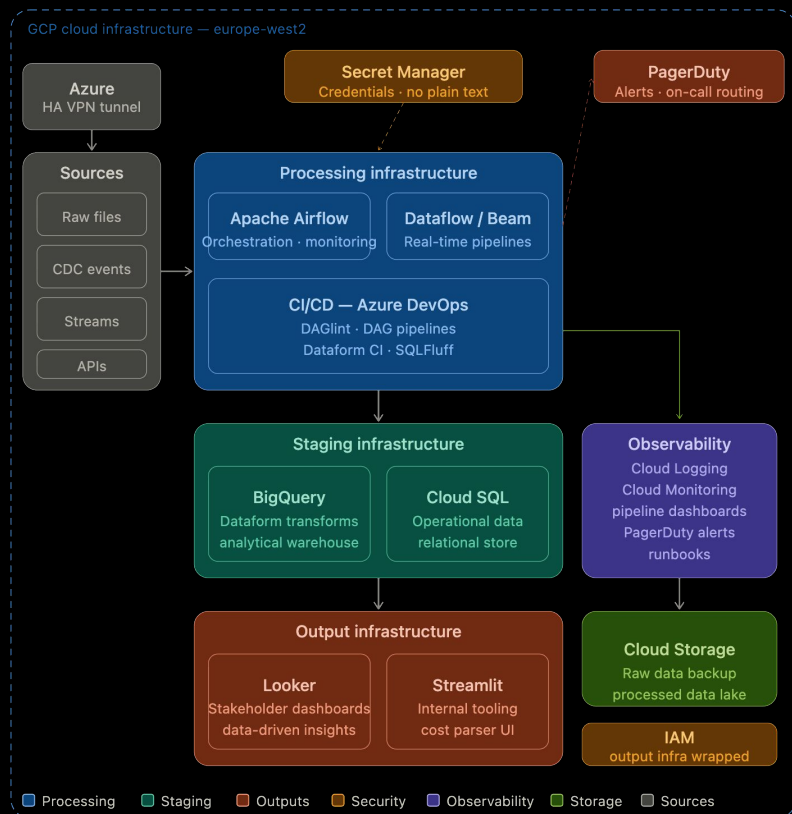
- Three main operational teams.
- ADE (Analytics and Data Engineering) sits across all the teams.
- Consume data from the Insignis machine and 3rd parties to deliver data driven insights.

Our constraints



- We are not a large data engineering team
- We do not have a big budget
- We do not have dedicated platform engineers

Our architecture



- Sources can have various formats – raw files, streams, APIs
- GCP tools used – Secret Manager for secrets, Dataform for SQL pipelines, Airflow for data pipelines and Dataflow for real-time infrastructure.
- Looker for client facing dashboards, Streamlit for internal apps, Cloud Monitoring for internal dashboards.

Set the relevant environment



- We use Composer 3 – handles scheduler, webserver and database automatically.
- Composer 3 has option to use the CeleryKubernetesExecutor.
- We have a small Composer environment. – will depend on number of DAGs, the number of concurrent tasks and how computationally intensive they are.
- Worker strategy: between 1 – 4 workers in the prod env.

Airflow best practice – Compute offloading



- As much as possible, use Airflow for orchestration, not an execution engine.
- If using APIs to fetch data – consider using sensible incremental strategies.
- Dataform for SQL/BigQuery orchestration, Dataflow for real-time pipelines, Dataproc for Spark.

Purge old logs and metadata



- Have a strategy in place to clean the Airflow logs and metadata table.
- Use a separate maintenance DAG or a Composer native strategy.

A screenshot of the Airflow web interface. On the left, a sidebar menu is partially visible with options like "Load snapshot", "Refresh", "Delete", "Environment variables", "Labels", and "Pypi packages". The main content area displays a modal dialog titled "Database data retention policy".

Database data retention policy

☒ Enable database data retention policy

Retention period * days

Your data will be permanently removed at the end of the retention period that you have specified.

Environment separation



- Have dev, test, staging environments much smaller than the prod one.
- Make the worker provisions very small in dev, test & staging.
- Consider turning off Composer env out of hours (evenings, weekends).

Our methods- GCP budget alerts



budget

← Edit Budget

Budget Alert

Name *
Budget Alert

Scope

A budget enables you to track your actual spend against your planned spend. Your costs are usually recorded within 24 hours. Set your budget to a lower amount to account for the time to report your costs. [Learn more](#)

Time range
Monthly

The month starts on the first of the month and resets at the beginning of each month.

☐ Read-only for project users (single-project budgets only)

Marking a budget read-only for project users restricts any inadvertent edits to important budgets tracked centrally.

A budget can be scoped to focus on a specific set of resources.

Folders & Organizations
All folders/organizations (2)

Projects
All projects (8)

Services
All services (1775)

Cost trend

July 1, 2025 – July 31, 2026

Month	Total cost (£K)	Forecasted additional cost (£K)
Jul	~3.5	~0.1
Aug	~1.2	~0.1
Sep	~1.2	~0.1
Oct	~1.2	~0.1
Nov	~1.1	~0.1
Dec	~1.2	~0.1
Jan	~1.3	~0.1
Feb	~1.4	~0.1
Mar	~1.5	~0.1
Apr	~1.5	~0.1
May	~1.8	~0.1
Jun	~1.8	~0.1
Jul	~2.2	~0.2

→ View report

Standard tooling – take advantage immediately.

Our methods- Maestro



 **Composer Tuner**

proud-stage-447812-g5-europe-west2-schedrigger

 Data loaded ✓

Overview

Composer version

composer-3-airfl...

Airflow version

unknown

DAGs

11

State

 **RUNNING**

Compute

Workers (Composer 2)

```
{
  "min_workers": 1
  "max_workers": 3
  "cpu": 0.5
  "memory_gb": 4
  "storage_gb": 10
}
```

Schedulers

```
{
  "count": 1
  "cpu": 0.5
  "memory_gb": 2
}
```

Airflow Config Overrides

```
{
  "webserver-show_trigger_form_if_no_params": "True"
}
```

Python packages

package	version
slack-sdk	==3.35.0
soda-core-bigquery	==3.5.6
google-cloud-dataform	==0.6.1
insights-airflow-common	==0.1.4
simple-salesforce	==1.12.9
soda-core	==3.5.6

Current metrics (latest value)

Active workers

—

Queue length

—

Pod evictions (24h)

—

DAG bag size

—

1. Compute

1.1 Scheduler — ⚠️ Underpowered

Parameter	Current	Recommended
<code>scheduler.cpu</code>	0.5 vCPU	2.0 vCPU
<code>scheduler.memoryGb</code>	2.0 GB	4.0 GB
<code>scheduler.count</code>	1	2 (HA mode)

Reasoning:

- The largest DAG (`mambu_recon_report_dag`) has 91 tasks. The scheduler must continuously parse, schedule, and heartbeat across all 10 DAGs (170+ tasks total). At 0.5 vCPU, the scheduler will become a bottleneck — it will struggle to keep up with DAG file parsing intervals and task queuing simultaneously.
- A single scheduler is a single point of failure. If it crashes or lags, no tasks are scheduled. Composer 3 supports multiple schedulers; running 2 provides redundancy with near-zero added risk.
- 2 GB RAM is marginal once the DagBag is fully loaded with Soda, Salesforce, and Dataform imports — these packages are heavy at parse time.

1.2 Workers — ⚠️ CPU Risk

Parameter	Current	Recommended
<code>worker.cpu</code>	0.5 vCPU	1.0–2.0 vCPU
<code>worker.memoryGb</code>	4.0 GB	4.0–6.0 GB (monitor first)
<code>worker.minCount</code>	1	2
<code>worker.maxCount</code>	3	5 (if peak concurrency requires)

Reasoning:

- At 0.5 vCPU per worker, tasks that do any meaningful computation (Soda data quality checks against BigQuery, Salesforce API calls, Mambu reconciliation logic) will run slowly and may hit timeout thresholds. Python GIL contention is less relevant but CPU starvation is real.
- With `minCount: 1`, a cold-start delay will occur whenever the single worker is fully occupied and autoscaling needs to provision a second worker (typically 2–5 minutes in Composer 3). For a development environment this may be acceptable, but it should be a conscious decision.
- 4 GB RAM is reasonable for current workloads but watch memory utilisation closely once metrics are available — `soda-core-bigquery` and `simple-salesforce` can be memory-hungry per task.

⚠️ OOM Risk Flag: `mambu_recon_report_dag` has 86 `PythonDecoratedOperator` tasks. If multiple tasks run concurrently on a single 4 GB worker (see Celery concurrency below), memory pressure could cause the worker pod to be OOM-killed. This is the highest-risk scenario in this environment.

Our methods- Tariff



Rendering lineage graph

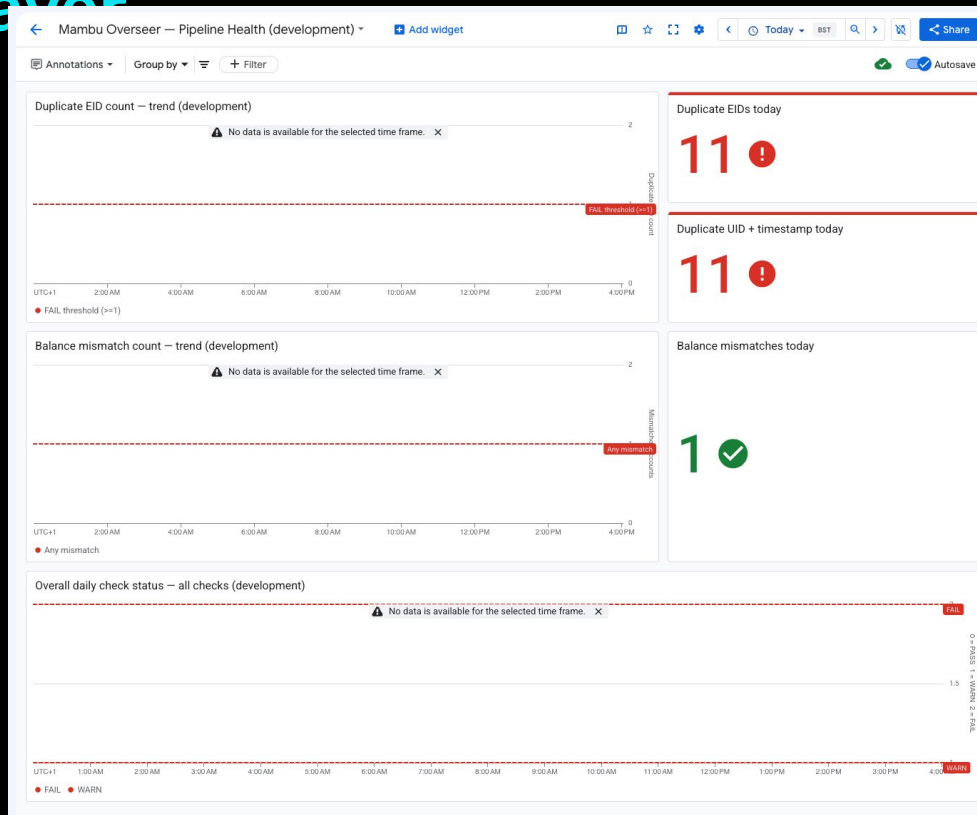


Total pipeline cost (USD, per run)

\$0.0106

- Internal Streamlit to app to evaluate the costs of running a Dataform pipeline.
- Using Dataform's built in json compiler and BigQuery's dry-run functionality.
- Gives us clear visibility on our Dataform pipelines.

Our methods- Airflow as an observability layer



- Airflow used an observation layer.
- Gives insight into health of pipeline.
- Dashboards hosted in Cloud Monitoring

What we would do differently



- Work on observability dashboards into pipeline health alongside the main data pipelines themselves instead of being an afterthought.
- Make more internal apps (Maestro) that helps to automate direct cost optimization – something that previously took a long time manually to do.
- Create and implement methods of automating governance sooner.

Takeaways



- Automation can be used intelligently as a substitution for headcount.
- You can't manage what you don't observe.
- Observability is cheaper than incidents.



Questions?

aniruddha.sengupta@insigniscash.com

[Company Linkedin](#)

[Personal Linkedin](#)